Sokoban Solver

Move a worker and boxes onto targets, and watch an A* search with deadlock detection compute and replay the solution.

What it is

A solver for Sokoban, the warehouse box-pushing puzzle, with a Tkinter window that animates the moves the AI works out. Written in Python 3 using **nothing but the standard library** -- there is no install step.

```
python main.py
```

Using it

Pick a level and press **Solve** (or Enter). The search runs on a background thread, reports how many positions it has examined, and replays the plan on the board when it finishes. **Space** pauses and resumes, **R** resets.

You can also play by hand with the **arrow keys** or **WASD** -- and Solve picks up from wherever you have got to, so you can push a few boxes yourself and let the AI finish.

Architecture

Module	Lines	Responsibility
sokoban/game.py	252	The rules: walls, targets, state, legal moves, push generation, .sok parsing
sokoban/solver.py	295	Pull distances, deadlock detection, Hungarian assignment, the A* loop
sokoban/gui.py	317	Tkinter window, threaded solving, replay timer, canvas drawing
main.py	7	Launches the GUI
tests/test_sokoban.py	157	25 tests

Dependencies run one way only: **gui -> solver -> game**. The game layer knows nothing about searching, and the solver knows nothing about the display, so it can be tested headlessly.

How the solver works

Sokoban is PSPACE-complete, so brute force dies quickly. Three things keep the search tractable.

1. It searches pushes, not footsteps

Where the worker walks between two pushes never changes what is solvable, so the search only branches on box pushes. The walking is reconstructed once at the end, so the GUI has a move sequence to animate. Cost is counted in pushes, the standard measure for Sokoban solvers.

2. Equivalent positions collapse into one

Two positions with identical boxes are the same node whenever the worker can walk between them, so a state is keyed by its boxes plus the top-left square of the worker's reachable area.

3. Lost positions are thrown out on sight

- **Simple deadlock** -- the box sits on a square from which no target can be reached: a corner, or the wall-hugging strip between two corners. One set lookup.
- **Freeze deadlock** -- the box can never move along either axis again, pinned by walls, dead squares or other immovable boxes. If it is not on a target, the position is lost. The check is mutually recursive, since box A can be frozen because B is and vice versa.

On the largest bundled level that discards 11,472 positions outright.

The heuristic

Before searching, the solver pulls a single imaginary box backwards out of each target to build a table of *pull distances*: how many pushes a lone box needs to get from any square to that target. Other boxes can only get in the way, so these are a lower bound on the true cost -- the heuristic is admissible and A* stays optimal.

Turning those distances into an estimate is an assignment problem, since every box needs its own target. Solving it exactly with the **Hungarian algorithm** costs O(n^3) in the number of boxes -- nothing next to the search, and worth it: letting each box pick its own nearest target instead expands 7.3x more nodes and takes 5.9x longer on the hardest bundled level.

Results

Level	Boxes	Pushes	Moves	Positions examined	Deadlocks pruned	Time
First Push	1	1	2	2	3	0.00s
Side by Side	2	2	7	3	5	0.00s
Around the Pillar	2	2	11	3	5	0.00s
The Hallway	2	6	27	7	6	0.00s
Long Way Round	1	9	22	10	7	0.00s
Crossroads	4	4	18	5	4	0.00s
The Vault	4	4	33	5	0	0.00s
Pinwheel	4	4	12	5	4	0.00s
Storage Room	4	20	78	36	57	0.01s
The Depot	3	21	79	39	43	0.00s
Tight Fit	4	10	49	11	38	0.00s
Warehouse Shift	5	28	117	5,539	11,472	2.01s

Measured when this document was generated. Every solution was replayed against the rules and verified.

Level format

levels/levels.sok, in the standard notation. Blank lines separate levels and a **;** comment above one titles it.

```
#  wall         @  player         +  player on target
$  box          *  box on target  .  target
```

Tests

```
python -m unittest discover -s tests -v
```

Covers parsing and validation, move legality, state normalisation, dead-square and freeze-deadlock detection, the assignment solver, and a solve-and-verify pass over every bundled level. Every solution is replayed against the rules, so the solver never marks its own homework.

Limits

- Cost is counted in pushes. The move string is the shortest walk between those pushes, not a move-optimal solution.
- Simple and freeze deadlocks only; corral deadlocks are not detected.
- Large levels (20+ boxes) will hit the 2,000,000-node / 60-second budget.

Appendix: full source

Every line of the project, as generated.

main.py

7 lines

```
#!/usr/bin/env python3
"""Run the Sokoban solver GUI:  python main.py"""

from sokoban.gui import run

if __name__ == "__main__":
    run()
```

sokoban/game.py

252 lines

```
"""The rules: walls, goals, state, legal moves, and .sok parsing."""

from __future__ import annotations

from collections import deque
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, FrozenSet, Iterator, List, Optional, Sequence, Set, Tuple

Pos = Tuple[int, int]

#: Row/column deltas, keyed by the letter used in LURD solution strings.
DIRECTIONS: Dict[str, Pos] = {"u": (-1, 0), "d": (1, 0), "l": (0, -1), "r": (0, 1)}

LEVELS_FILE = Path(__file__).resolve().parent.parent / "levels" / "levels.sok"


class LevelError(ValueError):
    """A level cannot be parsed into something playable."""


@dataclass(frozen=True)
class State:
    """Everything that changes while a level is played."""

    player: Pos
    boxes: FrozenSet[Pos]


@dataclass(frozen=True)
class Push:
    """The box on ``box_from`` moves one step along ``direction``."""

    box_from: Pos
    direction: str

    @property
    def box_to(self) -> Pos:
        dr, dc = DIRECTIONS[self.direction]
        return (self.box_from[0] + dr, self.box_from[1] + dc)

    @property
    def player_from(self) -> Pos:
        dr, dc = DIRECTIONS[self.direction]
        return (self.box_from[0] - dr, self.box_from[1] - dc)


class Board:
    """The fixed half of a level, plus move generation over it."""

    def __init__(self, rows: Sequence[str], name: str = "unnamed") -> None:
        if not rows:
            raise LevelError("level is empty")
        self.name = name
        self.height = len(rows)
        self.width = max(len(row) for row in rows)
        rows = [row.ljust(self.width) for row in rows]  # .sok rows are often ragged

        walls: Set[Pos] = set()
        goals: Set[Pos] = set()
```

```
        boxes: List[Pos] = []
        player: Optional[Pos] = None

        for r, row in enumerate(rows):
            for c, char in enumerate(row):
                pos = (r, c)
                if char == "#":
                    walls.add(pos)
                elif char in ".*+":
                    goals.add(pos)
                elif char not in "$@ -_":
                    raise LevelError(f"unexpected character {char!r} at row {r}, column {c}")
                if char in "$*":
                    boxes.append(pos)
                if char in "@+":
                    if player is not None:
                        raise LevelError("level has more than one player")
                    player = pos

        if player is None:
            raise LevelError("level has no player")
        if not boxes:
            raise LevelError("level has no boxes")
        if len(boxes) > len(goals):
            raise LevelError(f"{len(boxes)} boxes but only {len(goals)} goals -- unsolvable")

        self.walls: FrozenSet[Pos] = frozenset(walls)
        self.goals: FrozenSet[Pos] = frozenset(goals)
        self.initial = State(player, frozenset(boxes))
        self.floors: FrozenSet[Pos] = frozenset(self._flood_fill(player))

        stranded = [b for b in boxes if b not in self.floors]
        if stranded:
            raise LevelError(f"box at {stranded[0]} is walled off from the player")

    def _flood_fill(self, start: Pos) -> Set[Pos]:
        """Squares the player could stand on if boxes were not in the way.

        Doubles as the "is this level enclosed?" check -- escaping the grid
        means the border has a hole and the player could walk off into nothing.
        """
        seen = {start}
        queue = deque([start])
        while queue:
            r, c = queue.popleft()
            for dr, dc in DIRECTIONS.values():
                nr, nc = neighbour = (r + dr, c + dc)
                if not (0 <= nr < self.height and 0 <= nc < self.width):
                    raise LevelError("level is not fully enclosed by walls")
                if neighbour in seen or neighbour in self.walls:
                    continue
                seen.add(neighbour)
                queue.append(neighbour)
        return seen

    def reachable(self, player: Pos, boxes: FrozenSet[Pos]) -> Set[Pos]:
        """Squares the player can walk to without pushing anything."""
        seen = {player}
        queue = deque([player])
        while queue:
            r, c = queue.popleft()
            for dr, dc in DIRECTIONS.values():
                neighbour = (r + dr, c + dc)
                if neighbour in seen or neighbour in self.walls or neighbour in boxes:
                    continue
                seen.add(neighbour)
                queue.append(neighbour)
        return seen

    def normalize(self, state: State) -> Tuple[Pos, FrozenSet[Pos]]:
        """Canonical key for the closed set.

        Two positions with the same boxes are the same node whenever the player
        can walk between them, so key on the top-left square of the player's
        reachable area instead of where they happen to stand.
        """
        return (min(self.reachable(state.player, state.boxes)), state.boxes)

    def pushes(self, state: State) -> Iterator[Tuple[Push, State]]:
        """Every legal push from ``state``, as ``(push, resulting state)``."""
        area = self.reachable(state.player, state.boxes)
        boxes = state.boxes
        for box in boxes:
```

```
            br, bc = box
            for direction, (dr, dc) in DIRECTIONS.items():
                if (br - dr, bc - dc) not in area:
                    continue
                destination = (br + dr, bc + dc)
                if destination in self.walls or destination in boxes:
                    continue
                # The player ends up where the box was.
                yield Push(box, direction), State(box, frozenset(boxes - {box} | {destination}))

    def step(self, state: State, direction: str) -> Optional[State]:
        """One keyboard move, pushing a box if one is in the way, else ``None``."""
        dr, dc = DIRECTIONS[direction]
        target = (state.player[0] + dr, state.player[1] + dc)
        if target in self.walls:
            return None
        if target in state.boxes:
            beyond = (target[0] + dr, target[1] + dc)
            if beyond in self.walls or beyond in state.boxes:
                return None
            return State(target, frozenset(state.boxes - {target} | {beyond}))
        return State(target, state.boxes)

    def is_solved(self, state: State) -> bool:
        return state.boxes <= self.goals

    def walk_path(self, start: Pos, target: Pos, boxes: FrozenSet[Pos]) -> Optional[str]:
        """Shortest walk between two squares, as lowercase LURD letters."""
        if start == target:
            return ""
        came_from: Dict[Pos, Tuple[Pos, str]] = {}
        seen = {start}
        queue = deque([start])
        while queue:
            current = queue.popleft()
            r, c = current
            for direction, (dr, dc) in DIRECTIONS.items():
                neighbour = (r + dr, c + dc)
                if neighbour in seen or neighbour in self.walls or neighbour in boxes:
                    continue
                seen.add(neighbour)
                came_from[neighbour] = (current, direction)
                if neighbour == target:
                    letters = []
                    node = neighbour
                    while node != start:
                        node, letter = came_from[node]
                        letters.append(letter)
                    return "".join(reversed(letters))
                queue.append(neighbour)
        return None

    def moves_for_pushes(self, state: State, pushes: Sequence[Push]) -> str:
        """Expand a push sequence into keystrokes the GUI can replay.

        Pushes are uppercase, walking is lowercase -- the usual LURD convention.
        """
        player = state.player
        boxes = state.boxes
        letters: List[str] = []
        for push in pushes:
            walk = self.walk_path(player, push.player_from, boxes)
            if walk is None:  # pragma: no cover - only reachable on a solver bug
                raise LevelError(f"cannot reach {push.player_from} to make {push}")
            letters.append(walk)
            letters.append(push.direction.upper())
            boxes = frozenset(boxes - {push.box_from} | {push.box_to})
            player = push.box_from
        return "".join(letters)


def parse_collection(text: str, source: str = "<string>") -> List[Board]:
    """Parse a .sok collection: levels split by blank lines, ``;`` lines are titles."""
    boards: List[Board] = []
    rows: List[str] = []
    title: Optional[str] = None
    pending: Optional[str] = None

    def flush() -> None:
        nonlocal rows, title
        if rows:
            name = title or f"Level {len(boards) + 1}"
            try:
```

```
                boards.append(Board(rows, name=name))
            except LevelError as error:
                raise LevelError(f"{source}: level {name!r} is invalid: {error}") from error
        rows = []
        title = None

    for line in text.splitlines():
        stripped = line.strip()
        if stripped.startswith(";"):
            flush()  # a comment ends the current level and titles the next
            pending = stripped.lstrip("; ").strip() or None
            continue
        if not stripped:
            flush()
            continue
        if not rows:
            title, pending = pending, None
        rows.append(line)

    flush()
    return boards


def load_levels() -> List[Board]:
    """The bundled level pack."""
    return parse_collection(LEVELS_FILE.read_text(encoding="utf-8"), source=str(LEVELS_FILE))
```

sokoban/solver.py

295 lines

```
"""A* over box pushes, with deadlock pruning.

Three things keep the search tractable:

* It branches on *box pushes*, not footsteps -- the walking between pushes
  never changes what is solvable, so it is reconstructed once at the end.
* Positions that differ only in where the player stands collapse to one node
  (see :meth:`Board.normalize`).
* Positions that are already lost are thrown out on sight (:class:`Analysis`).

Cost is counted in pushes, the standard measure for Sokoban solvers.
"""

from __future__ import annotations

import time
from collections import deque
from dataclasses import dataclass, field
from heapq import heappop, heappush
from typing import Callable, Dict, FrozenSet, List, Optional, Sequence, Set, Tuple

from .game import DIRECTIONS, Board, Pos, Push, State

#: Stands in for "no path exists".  Finite, so the matching below can use plain
#: integer arithmetic.
UNREACHABLE = 1_000_000

_AXES = (((0, -1), (0, 1)), ((-1, 0), (1, 0)))  # horizontal, vertical


class Analysis:
    """Per-level facts worked out once, before the search starts.

    All of it comes from pretending there is a single box on the board and
    *pulling* it backwards out of each goal -- the reverse of pushing it in.
    That gives, for every square, how many pushes a lone box needs to reach
    that goal.  Other boxes can only ever get in the way, so those numbers are
    a lower bound on the real cost and the heuristic built from them is
    admissible.
    """

    def __init__(self, board: Board) -> None:
        self.board = board
        self.goals: List[Pos] = sorted(board.goals)
        self.pull_distance = {goal: self._pull_distances_from(goal) for goal in self.goals}
        reachable_from_some_goal: Set[Pos] = set()
        for table in self.pull_distance.values():
            reachable_from_some_goal |= table.keys()
        #: Squares no goal can be reached from: corners, and the wall-hugging
        #: strips between two corners.  A box here can never be recovered.
        self.dead_squares = frozenset(board.floors - reachable_from_some_goal)
```

```

    def _pull_distances_from(self, goal: Pos) -> Dict[Pos, int]:
        # Pulling a box one square needs the square behind it and the one
        # behind that to be free, because the player walks backwards into them.
        distances = {goal: 0}
        queue = deque([goal])
        while queue:
            r, c = current = queue.popleft()
            for dr, dc in DIRECTIONS.values():
                box_to = (r + dr, c + dc)
                if box_to in distances or box_to not in self.board.floors:
                    continue
                if (r + 2 * dr, c + 2 * dc) in self.board.walls:
                    continue
                distances[box_to] = distances[current] + 1
                queue.append(box_to)
        return distances

    def distance(self, box: Pos, goal: Pos) -> int:
        return self.pull_distance[goal].get(box, UNREACHABLE)

    def heuristic(self, state: State) -> int:
        """Pushes still needed, lower bound.

        Every box needs its own goal, which makes this an assignment problem;
        solving it exactly beats letting each box pick its nearest goal by
        roughly 6x in nodes expanded.  Boxes already on a goal are included
        rather than pinned there, because an optimal solution is allowed to
        push one off and bring it back.
        """
        boxes = sorted(state.boxes)
        return min_cost_assignment(
            [[self.distance(box, goal) for goal in self.goals] for box in boxes]
        )

    def is_deadlock(self, state: State, last_box: Optional[Pos] = None) -> bool:
        """Whether this position is already lost.

        ``last_box`` is the box that just moved -- only it can have created a
        new deadlock, so passing it turns a whole-board scan into one check.
        """
        boxes = state.boxes
        for box in (last_box,) if last_box is not None else tuple(boxes):
            if box in self.board.goals:
                continue
            if box in self.dead_squares:
                return True
            if self._is_frozen(box, boxes, set()):
                return True
        return False

    def _is_frozen(self, box: Pos, boxes: FrozenSet[Pos], assumed: Set[Pos]) -> bool:
        """Whether ``box`` can never move along either axis again."""
        if box in assumed:
            # Box A can be frozen because B is and vice versa.  Assuming it
            # frozen while its own check runs terminates the recursion safely:
            # if the assumption is wrong the outer call reports not-frozen.
            return True
        assumed.add(box)
        try:
            return all(self._axis_blocked(box, axis, boxes, assumed) for axis in _AXES)
        finally:
            assumed.discard(box)

    def _axis_blocked(self, box, axis, boxes: FrozenSet[Pos], assumed: Set[Pos]) -> bool:
        r, c = box
        sides = [(r + dr, c + dc) for dr, dc in axis]
        if any(side in self.board.walls for side in sides):
            return True
        # Both sides dead: the box could move, but only onto a square it can
        # never come back from, so the position is lost either way.
        if all(side in self.dead_squares for side in sides):
            return True
        return any(side in boxes and self._is_frozen(side, boxes, assumed) for side in sides)


def min_cost_assignment(cost: Sequence[Sequence[int]]) -> int:
    """Cheapest way to give every row its own column (Hungarian algorithm).

    O(n^3) shortest-augmenting-path form with potentials.  Rows are boxes and
    columns are goals; there are never more boxes than goals, so the matrix may
    be wider than tall.  Indices are 1-based, with 0 as the path sentinel.
    """
```

```
    rows = len(cost)
    if rows == 0:
        return 0
    cols = len(cost[0])
    big = UNREACHABLE * (rows + cols + 1)
    u = [0] * (rows + 1)
    v = [0] * (cols + 1)
    match = [0] * (cols + 1)  # column -> row
    parent = [0] * (cols + 1)  # column -> previous column on the path

    for row in range(1, rows + 1):
        match[0] = row
        col = 0
        slack = [big] * (cols + 1)
        used = [False] * (cols + 1)
        while True:
            used[col] = True
            current_row = match[col]
            delta, next_col = big, 0
            for j in range(1, cols + 1):
                if used[j]:
                    continue
                reduced = cost[current_row - 1][j - 1] - u[current_row] - v[j]
                if reduced < slack[j]:
                    slack[j], parent[j] = reduced, col
                if slack[j] < delta:
                    delta, next_col = slack[j], j
            for j in range(cols + 1):
                if used[j]:
                    u[match[j]] += delta
                    v[j] -= delta
                else:
                    slack[j] -= delta
            col = next_col
            if match[col] == 0:
                break
        while col:  # walk the augmenting path back, flipping pairs
            match[col], col = match[parent[col]], parent[col]

    return sum(cost[match[j] - 1][j - 1] for j in range(1, cols + 1) if match[j])


@dataclass
class Result:
    """Outcome of one solve.

    ``status`` is ``solved``, ``unsolvable`` (the whole reachable state space
    was exhausted) or ``aborted`` (a limit was hit or the caller cancelled,
    which says nothing about whether a solution exists).
    """

    status: str
    moves: str = ""
    pushes: int = 0
    nodes: int = 0
    pruned: int = 0
    elapsed: float = 0.0

    @property
    def solved(self) -> bool:
        return self.status == "solved"

    def summary(self) -> str:
        return f"{self.nodes:,} nodes, {self.pruned:,} deadlocks pruned, {self.elapsed:.2f}s"


def solve(
    board: Board,
    state: Optional[State] = None,
    cancel: Optional[Callable[[], bool]] = None,
    on_progress: Optional[Callable[[Result], None]] = None,
    max_nodes: int = 2_000_000,
    max_seconds: float = 60.0,
) -> Result:
    """A* from ``state`` (the level's start position by default)."""
    state = board.initial if state is None else state
    analysis = Analysis(board)
    started = time.perf_counter()
    result = Result("aborted")

    def finish(status: str, pushes: Sequence[Push] = ()) -> Result:
        result.status = status
        result.elapsed = time.perf_counter() - started
```

```
        result.pushes = len(pushes)
        result.moves = board.moves_for_pushes(state, pushes) if pushes else ""
        return result

    if board.is_solved(state):
        return finish("solved")
    if analysis.is_deadlock(state):
        return finish("unsolvable")

    start_key = board.normalize(state)
    came_from: Dict[object, Optional[Tuple[object, Push]]] = {start_key: None}
    best_cost: Dict[object, int] = {start_key: 0}
    closed: Set[object] = set()
    counter = 0
    # Ties on f are broken by lower h: among equally promising nodes, go for
    # the one nearest the goal first.  Worth ~5x in nodes expanded.
    start_h = analysis.heuristic(state)
    frontier = [(start_h, start_h, counter, state)]

    while frontier:
        if result.nodes % 2000 == 0:
            result.elapsed = time.perf_counter() - started
            if (cancel is not None and cancel()) or result.elapsed > max_seconds:
                return finish("aborted")
            if on_progress is not None and result.nodes:
                on_progress(result)
        if result.nodes >= max_nodes:
            return finish("aborted")

        _, _, _, current = heappop(frontier)
        key = board.normalize(current)
        if key in closed:
            continue
        closed.add(key)
        result.nodes += 1

        if board.is_solved(current):
            return finish("solved", _reconstruct(came_from, key))

        cost = best_cost[key] + 1
        for push, successor in board.pushes(current):
            if analysis.is_deadlock(successor, push.box_to):
                result.pruned += 1
                continue
            estimate = analysis.heuristic(successor)
            if estimate >= UNREACHABLE:
                result.pruned += 1  # some box has no goal it can still reach
                continue
            successor_key = board.normalize(successor)
            if successor_key in closed or best_cost.get(successor_key, cost + 1) <= cost:
                continue
            best_cost[successor_key] = cost
            came_from[successor_key] = (key, push)
            counter += 1
            heappush(frontier, (cost + estimate, estimate, counter, successor))

    return finish("unsolvable")


def _reconstruct(came_from, key) -> List[Push]:
    pushes: List[Push] = []
    entry = came_from[key]
    while entry is not None:
        parent_key, push = entry
        pushes.append(push)
        entry = came_from[parent_key]
    pushes.reverse()
    return pushes


def verify(board: Board, moves: str, state: Optional[State] = None) -> bool:
    """Replay a move string against the rules and report whether it solves the level."""
    current = board.initial if state is None else state
    for letter in moves:
        current = board.step(current, letter.lower())
        if current is None:
            return False
    return board.is_solved(current)
```

sokoban/gui.py

317 lines

```
"""The window: push boxes around by hand, or watch the solver's plan replay.

The search runs on a worker thread and reports through a queue the Tk main loop
drains on a timer, so the window stays responsive on hard levels and Stop
actually stops.
"""

from __future__ import annotations

import queue
import threading
import tkinter as tk
from tkinter import ttk
from typing import List, Optional

from .game import Board, State, load_levels
from .solver import Result, solve

COLOURS = {
    "bg": "#15171c",
    "panel": "#1d2027",
    "text": "#d7dae1",
    "muted": "#8b93a3",
    "wall": "#39404f",
    "wall_edge": "#4a5365",
    "floor": "#242832",
    "goal": "#d8a33b",
    "box": "#c0783c",
    "box_done": "#4f9d58",
    "player": "#4f8ef7",
}

REPLAY_DELAY_MS = 90
KEYS = {"Up": "u", "Down": "d", "Left": "l", "Right": "r",
        "w": "u", "s": "d", "a": "l", "d": "r"}


class SokobanApp:
    def __init__(self, root: tk.Tk, boards: List[Board]) -> None:
        self.root = root
        self.boards = boards
        self.board = boards[0]
        self.state: State = self.board.initial
        self.moves_made = 0

        self.solution = ""
        self.solution_index = 0
        self.playing = False
        self._tick_id: Optional[str] = None

        self.results: "queue.Queue[Result]" = queue.Queue()
        self.cancelled = threading.Event()
        self.solving = False
        self.progress = 0  # written by the worker, read by the Tk timer

        self.status = tk.StringVar()
        self._build_ui()
        self.load_level(0)
        self.root.after(60, self._poll_solver)

    # ------------------------------------------------------------------ ui

    def _build_ui(self) -> None:
        root = self.root
        root.title("Sokoban Solver -- A* with deadlock detection")
        root.configure(bg=COLOURS["bg"])
        root.minsize(720, 520)

        style = ttk.Style()
        try:
            style.theme_use("clam")
        except tk.TclError:  # pragma: no cover - platform dependent
            pass
        style.configure("TFrame", background=COLOURS["panel"])
        style.configure("TLabel", background=COLOURS["panel"], foreground=COLOURS["text"])
        style.configure("TButton", padding=(10, 5))

        sidebar = ttk.Frame(root, padding=14)
        sidebar.pack(side="left", fill="y")

        ttk.Label(sidebar, text="Level").pack(anchor="w")
        self.level_box = ttk.Combobox(
            sidebar, state="readonly", width=22, values=[b.name for b in self.boards]
```

```
        )
        self.level_box.pack(anchor="w", pady=(2, 16))
        self.level_box.bind(
            "<<ComboboxSelected>>", lambda _: self.load_level(self.level_box.current())
        )

        self.solve_button = ttk.Button(sidebar, text="Solve  (Enter)", command=self.solve)
        self.solve_button.pack(fill="x", pady=(0, 6))
        self.play_button = ttk.Button(sidebar, text="Play  (Space)", command=self.toggle_play)
        self.play_button.pack(fill="x", pady=(0, 6))
        ttk.Button(sidebar, text="Reset  (R)", command=self.reset).pack(fill="x")

        ttk.Label(
            sidebar, foreground=COLOURS["muted"], justify="left", wraplength=170,
            text="Arrow keys or WASD to push boxes yourself. Solve continues from "
                 "wherever you have got to.",
        ).pack(anchor="w", pady=(20, 0))

        area = tk.Frame(root, bg=COLOURS["bg"])
        area.pack(side="right", fill="both", expand=True)
        self.canvas = tk.Canvas(area, bg=COLOURS["bg"], highlightthickness=0)
        self.canvas.pack(fill="both", expand=True, padx=12, pady=(12, 4))
        self.canvas.bind("<Configure>", lambda _: self.draw())
        tk.Label(
            area, textvariable=self.status, bg=COLOURS["bg"], fg=COLOURS["text"],
            anchor="w", font=("Segoe UI", 10),
        ).pack(fill="x", padx=16, pady=(0, 12))

        root.bind("<Key>", self.on_key)

    # ------------------------------------------------------------- state

    def load_level(self, index: int) -> None:
        self.cancelled.set()
        index = max(0, min(index, len(self.boards) - 1))
        self.board = self.boards[index]
        self.level_box.current(index)
        self.reset()

    def reset(self) -> None:
        self.stop_playing()
        self.state = self.board.initial
        self.moves_made = 0
        self.solution = ""
        self.solution_index = 0
        self.draw()
        self.report()

    def do_move(self, direction: str) -> bool:
        nxt = self.board.step(self.state, direction)
        if nxt is None:
            return False
        self.state = nxt
        self.moves_made += 1
        self.draw()
        self.report()
        return True

    def on_key(self, event: tk.Event) -> None:
        key = event.keysym
        direction = KEYS.get(key) or KEYS.get(key.lower())
        if direction:
            self.stop_playing()
            self.solution = ""  # the plan no longer lines up with the position
            self.do_move(direction)
        elif key.lower() == "r":
            self.reset()
        elif key in ("Return", "KP_Enter"):
            self.solve()
        elif key == "space":
            self.toggle_play()

    def report(self) -> None:
        if self.solving:
            return
        placed = len(self.state.boxes & self.board.goals)
        line = f"{placed}/{len(self.state.boxes)} boxes on target  ·  {self.moves_made} moves"
        if self.solution:
            line += f"  ·  replaying {self.solution_index}/{len(self.solution)}"
        self.status.set("Solved!  " + line if self.board.is_solved(self.state) else line)

    # ------------------------------------------------------------- solving

```

```
    def solve(self) -> None:
        if self.solving:
            self.cancelled.set()
            return
        self.stop_playing()
        self.solution = ""
        self.solving = True
        self.cancelled.clear()
        self.solve_button.configure(text="Stop")
        self.status.set("Searching…")

        board, state = self.board, self.state

        def run() -> None:
            # Touching Tk off the main thread is not safe, so just park the
            # count where the timer below can pick it up.
            self.results.put(
                solve(
                    board, state,
                    cancel=self.cancelled.is_set,
                    on_progress=lambda partial: setattr(self, "progress", partial.nodes),
                )
            )

        threading.Thread(target=run, daemon=True).start()

    def _poll_solver(self) -> None:
        try:
            while True:
                self._solved(self.results.get_nowait())
        except queue.Empty:
            if self.solving and self.progress:
                self.status.set(f"Searching… {self.progress:,} positions examined")
        self.root.after(60, self._poll_solver)

    def _solved(self, result: Result) -> None:
        self.solving = False
        self.progress = 0
        self.solve_button.configure(text="Solve  (Enter)")
        if self.cancelled.is_set():
            # Stopped by the user, or the level changed under it.
            self.report()
            return
        if result.solved:
            self.solution = result.moves
            self.solution_index = 0
            self.status.set(
                f"Solved in {result.pushes} pushes, {len(result.moves)} moves"
                f"  ·  {result.summary()}"
            )
            self.start_playing()
        elif result.status == "unsolvable":
            self.status.set(f"No solution from this position  ·  {result.summary()}")
        else:
            self.status.set(f"Stopped  ·  {result.summary()}")

    # ---------------------------------------------------------------- replay

    def toggle_play(self) -> None:
        if self.playing:
            self.stop_playing()
        else:
            self.start_playing()

    def start_playing(self) -> None:
        if self.solution_index >= len(self.solution):
            return
        self.playing = True
        self.play_button.configure(text="Pause  (Space)")
        self._tick()

    def stop_playing(self) -> None:
        self.playing = False
        if self._tick_id is not None:
            self.root.after_cancel(self._tick_id)
            self._tick_id = None
        self.play_button.configure(text="Play  (Space)")

    def _tick(self) -> None:
        if not self.playing or not self.step_forward():
            self.stop_playing()
            return
        self._tick_id = self.root.after(REPLAY_DELAY_MS, self._tick)
```

```
250
251     def step_forward(self) -> bool:
252         if self.solution_index >= len(self.solution):
253             return False
254         if not self.do_move(self.solution[self.solution_index].lower()):
255             return False  # pragma: no cover - solver bug guard
256         self.solution_index += 1
257         self.report()  # do_move reported before the index moved
258         return True
259
260     # ------------------------------------------------------------ drawing
261
262     def draw(self) -> None:
263         canvas = self.canvas
264         canvas.delete("all")
265         width, height = canvas.winfo_width(), canvas.winfo_height()
266         if width < 10 or height < 10:
267             return
268
269         board = self.board
270         cell = max(8, min(width // board.width, height // board.height))
271         x0 = (width - cell * board.width) // 2
272         y0 = (height - cell * board.height) // 2
273
274         for r in range(board.height):
275             for c in range(board.width):
276                 pos = (r, c)
277                 left, top = x0 + c * cell, y0 + r * cell
278                 if pos in board.walls:
279                     canvas.create_rectangle(
280                         left, top, left + cell, top + cell,
281                         fill=COLOURS["wall"], outline=COLOURS["wall_edge"],
282                     )
283                     continue
284                 if pos not in board.floors:
285                     continue  # outside the playable area
286                 canvas.create_rectangle(
287                     left, top, left + cell, top + cell, fill=COLOURS["floor"], outline=""
288                 )
289                 if pos in board.goals:
290                     inset = cell // 3
291                     canvas.create_oval(
292                         left + inset, top + inset, left + cell - inset, top + cell - inset,
293                         outline=COLOURS["goal"], width=max(2, cell // 14),
294                     )
295
296         pad = max(2, cell // 10)
297         for r, c in self.state.boxes:
298             left, top = x0 + c * cell + pad, y0 + r * cell + pad
299             canvas.create_rectangle(
300                 left, top, left + cell - 2 * pad, top + cell - 2 * pad,
301                 fill=COLOURS["box_done"] if (r, c) in board.goals else COLOURS["box"],
        outline="",
302             )
303
304         r, c = self.state.player
305         inset = max(3, cell // 6)
306         canvas.create_oval(
307             x0 + c * cell + inset, y0 + r * cell + inset,
308             x0 + (c + 1) * cell - inset, y0 + (r + 1) * cell - inset,
309             fill=COLOURS["player"], outline="",
310         )
311
312
313 def run() -> None:
314     root = tk.Tk()
315     root.geometry("900x620")
316     SokobanApp(root, load_levels())
317     root.mainloop()
```

tests/test_sokoban.py

157 lines

```
  1 """Tests.  Run with ``python -m unittest discover -s tests -v``."""
  2
  3 from __future__ import annotations
  4
  5 import sys
  6 import unittest
  7 from pathlib import Path
  8
```

```
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from sokoban.game import Board, LevelError, State, load_levels, parse_collection  # noqa: E402
from sokoban.solver import UNREACHABLE, Analysis, min_cost_assignment, solve, verify  # noqa:
        E402

TRIVIAL = "#####\n#@  #\n# $.#\n#   #\n#####"
CORNER_TRAP = "#######\n#@    #\n#  $  #\n#    .#\n#######"


def level(text: str) -> Board:
    return parse_collection(text)[0]


class ParsingTests(unittest.TestCase):
    def test_parses_pieces(self) -> None:
        board = level(TRIVIAL)
        self.assertEqual(board.initial.player, (1, 1))
        self.assertEqual(board.initial.boxes, frozenset({(2, 2)}))
        self.assertEqual(board.goals, frozenset({(2, 3)}))

    def test_player_on_goal_counts_as_a_goal(self) -> None:
        board = level("#####\n#$+.#\n#####")
        self.assertEqual(board.goals, frozenset({(1, 2), (1, 3)}))
        self.assertEqual(board.initial.player, (1, 2))

    def test_rejects_unenclosed_level(self) -> None:
        with self.assertRaises(LevelError):
            level("###\n#@$.\n###")

    def test_rejects_missing_player(self) -> None:
        with self.assertRaises(LevelError):
            level("#####\n# $.#\n#####")

    def test_rejects_more_boxes_than_goals(self) -> None:
        with self.assertRaises(LevelError):
            level("######\n#@$$.#\n######")

    def test_bundled_pack_loads_with_unique_titles(self) -> None:
        boards = load_levels()
        self.assertGreaterEqual(len(boards), 12)
        self.assertEqual(len({b.name for b in boards}), len(boards))


class MovementTests(unittest.TestCase):
    def setUp(self) -> None:
        self.board = level(TRIVIAL)

    def test_walking_into_a_wall_is_blocked(self) -> None:
        self.assertIsNone(self.board.step(self.board.initial, "u"))

    def test_pushing_moves_both(self) -> None:
        pushed = self.board.step(self.board.step(self.board.initial, "d"), "r")
        self.assertEqual(pushed.player, (2, 2))
        self.assertEqual(pushed.boxes, frozenset({(2, 3)}))
        self.assertTrue(self.board.is_solved(pushed))

    def test_generated_pushes_are_legal(self) -> None:
        for push, successor in self.board.pushes(self.board.initial):
            self.assertIn(push.box_from, self.board.initial.boxes)
            self.assertIn(push.box_to, successor.boxes)
            self.assertNotIn(push.box_to, self.board.walls)

    def test_normalisation_collapses_equivalent_player_squares(self) -> None:
        boxes = self.board.initial.boxes
        self.assertEqual(
            self.board.normalize(State((1, 1), boxes)),
            self.board.normalize(State((3, 3), boxes)),
        )


class AnalysisTests(unittest.TestCase):
    def test_corners_are_dead_squares(self) -> None:
        analysis = Analysis(level(CORNER_TRAP))
        self.assertIn((1, 1), analysis.dead_squares)
        self.assertNotIn((2, 5), analysis.dead_squares)

    def test_distances_from_a_goal(self) -> None:
        board = level(CORNER_TRAP)
        analysis = Analysis(board)
        goal = next(iter(board.goals))
        self.assertEqual(analysis.distance(goal, goal), 0)
        self.assertEqual(analysis.distance((1, 1), goal), UNREACHABLE)
```

```

    def test_box_in_a_corner_is_a_deadlock(self) -> None:
        board = level(CORNER_TRAP)
        self.assertTrue(Analysis(board).is_deadlock(State((2, 2), frozenset({(1, 1)}))))

    def test_starting_position_is_fine(self) -> None:
        board = level(CORNER_TRAP)
        self.assertFalse(Analysis(board).is_deadlock(board.initial))

    def test_two_boxes_freezing_each_other_against_a_wall(self) -> None:
        board = level("######\n#@   #\n# $$ #\n#  . #\n#  . #\n######")
        frozen = State((2, 1), frozenset({(1, 2), (1, 3)}))
        self.assertTrue(Analysis(board).is_deadlock(frozen))

    def test_box_on_a_goal_is_never_a_deadlock(self) -> None:
        board = level("#####\n#@*.#\n#####")
        self.assertFalse(Analysis(board).is_deadlock(board.initial))


class AssignmentTests(unittest.TestCase):
    def test_avoids_the_greedy_trap(self) -> None:
        # Both rows want column 0; the real optimum pays 3 to free it up.
        self.assertEqual(min_cost_assignment([[1, 3], [1, 100]]), 4)

    def test_handles_more_columns_than_rows(self) -> None:
        self.assertEqual(min_cost_assignment([[5, 2, 9]]), 2)

    def test_empty_matrix(self) -> None:
        self.assertEqual(min_cost_assignment([]), 0)


class SolverTests(unittest.TestCase):
    def test_solves_verifies_and_stays_admissible_on_every_level(self) -> None:
        for board in load_levels():
            with self.subTest(level=board.name):
                result = solve(board)
                self.assertEqual(result.status, "solved")
                self.assertTrue(verify(board, result.moves))
                # Admissible: the start estimate cannot exceed the optimal
                # push count A* actually found.
                self.assertLessEqual(Analysis(board).heuristic(board.initial), result.pushes)

    def test_reports_unsolvable_rather_than_hanging(self) -> None:
        board = level("#######\n#@    #\n#   # #\n#  $# #\n#   #.#\n#######")
        self.assertEqual(solve(board).status, "unsolvable")

    def test_already_solved_position_needs_no_pushes(self) -> None:
        result = solve(level("#####\n#@*.#\n#####"))
        self.assertTrue(result.solved)
        self.assertEqual(result.pushes, 0)

    def test_search_stays_efficient(self) -> None:
        # Node counts are deterministic, so this is a real regression guard:
        # dropping the f-tie-break on h quietly costs about 5x here.
        result = solve(load_levels()[11])
        self.assertEqual(result.pushes, 28)
        self.assertLess(result.nodes, 10_000)

    def test_node_limit_aborts(self) -> None:
        self.assertEqual(solve(load_levels()[11], max_nodes=20).status, "aborted")

    def test_verify_rejects_a_wrong_answer(self) -> None:
        self.assertFalse(verify(level(TRIVIAL), "dd"))


if __name__ == "__main__":
    unittest.main()
```

levels/levels.sok

114 lines

```
; Sokoban level pack
; Ten original levels, roughly in order of difficulty.
; Legend:  # wall   @ player   + player on goal   $ box   * box on goal   . goal

; 1. First Push
#####
#@  #
# $.#
#   #
#####
```

```
11
12  ; 2. Side by Side
13  ######
14  #    #
15  # $$ #
16  # ..@#
17  ######
18
19  ; 3. Around the Pillar
20  #######
21  #     #
22  # $#$ #
23  # . . #
24  #  @  #
25  #######
26
27  ; 4. The Hallway
28  ########
29  #      #
30  # #  # #
31  # $  $ #
32  # .. @ #
33  ########
34
35  ; 5. Long Way Round
36  #########
37  #   #   #
38  # $ #.  #
39  #   #   #
40  #  ## ###
41  #       #
42  #   @   #
43  #########
44
45  ; 6. Crossroads
46  #######
47  #  .  #
48  # $$$ #
49  #..@..#
50  #  $  #
51  #  .  #
52  #######
53
54  ; 7. The Vault
55  ########
56  #      #
57  #  ##  #
58  # $..$ #
59  # $..$ #
60  #  ##  #
61  #  @   #
62  ########
63
64  ; 8. Pinwheel
65  ########
66  # .  . #
67  # $##$ #
68  #  @   #
69  # $##$ #
70  # .  . #
71  ########
72
73  ; 9. Storage Room
74  ##########
75  #        #
76  #  $$$$  #
77  #        #
78  ### ## ###
79  #        #
80  #  ....  #
81  #   @    #
82  ##########
83
84  ; 10. The Depot
85  ##########
86  #  #     #
87  #  # $$$ #
88  #  #     #
89  #  ## ####
90  #        #
91  # ...    #
92  #    @   #
93  ##########
```

```
 94
 95  ; 11. Tight Fit
 96  #########
 97  #   .   #
 98  # $ . $ #
 99  #  ###  #
100  # $ . $ #
101  #   .   #
102  #   @   #
103  #########
104
105  ; 12. Warehouse Shift
106  ############
107  #          #
108  #  $$$$$   #
109  #          #
110  #### # ####
111  #          #
112  #  .....   #
113  #    @     #
114  ############
```